

How KV Cache and PagedAttention Speed Up LLM Inference

Understand why serving an LLM to many users is a memory problem, and how KV cache, PagedAttention, and vLLM's tuning knobs turn wasted GPU memory into throughput.

For engineers who deploy or serve LLMs in production and want to understand where GPU memory and latency actually go · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

[Watch the source video](#)[Read the full lesson](#)[Read the condensed version](#)

What matters

- ✓ LLM inference has two distinct phases: prefill (compute-bound, processes the whole prompt at once) and decode (memory-bound, generates one token at a time).
- ✓ The KV cache avoids recomputing attention keys and values for every prior token at each generation step, trading GPU memory for compute.
- ✓ Model weights alone can consume the majority of a GPU's memory, leaving a small, fixed budget that every concurrent request's KV cache has to share.
- ✓ Naive fixed-block KV cache allocation reserves memory for the worst-case sequence length per request, and research cited in the video puts the resulting waste at 60-80% of the KV cache budget.
- ✓ PagedAttention applies OS-style virtual memory paging — small fixed-size, non-contiguous pages plus a block table — to eliminate internal fragmentation, external fragmentation, and duplicate storage of shared prefixes.
- ✓ Prefix caching and chunked prefill are practical vLLM settings that reuse KV cache across requests sharing a prompt and stop long prefills from stalling in-flight decode work, respectively.
- ✓ Speculative decoding spends otherwise-idle GPU compute during decode to draft and verify multiple tokens per step, cutting latency without changing output quality, and is most useful at low-to-moderate concurrency.

What it covers

- 01 Why Long Prompts Feel Slow: The Two Phases of LLM Inference** 0:00
Serving an LLM exposes two different bottlenecks: a compute-bound prefill phase before the first token, and a memory-bound decode phase for every token after that.
- 02 The KV Cache: Trading Memory for Compute** 2:53
Attention needs a query, key, and value for every token at every layer; caching the keys and values means each new token only computes its own instead of recomputing history.
- 03 How Naive KV Cache Allocation Wastes GPU Memory** 4:20
Weights alone can eat most of a GPU's memory, and traditional serving systems reserve a fixed worst-case block of the remainder for every request, wasting most of it.
- 04 PagedAttention: Virtual Memory for the GPU** 5:48
PagedAttention borrows the OS trick of paging: it splits KV cache into small fixed-size blocks that can live anywhere in memory, addressed through a lightweight lookup table.
- 05 Tuning vLLM for Maximum Throughput** 7:56

Three practical vLLM settings, GPU memory utilization, prefix caching, and chunked prefill, turn PagedAttention's efficiency into measurable throughput and latency gains.

06 **Speculative Decoding: Spending Idle GPU Cycles** 9:24

During decode the GPU has spare compute sitting idle between memory reads; speculative decoding uses that slack to draft several tokens at once and verify them in a single pass.

Glossary

Prefill phase — The compute-bound stage of LLM inference where the model processes the entire input prompt at once to build its internal representation before generating any output token.

KV cache — Stored attention key and value vectors from previously processed tokens, kept in GPU memory so later tokens do not have to recompute them.

Causal attention — An attention pattern where a token can only attend to itself and the tokens before it, never to tokens that come later.

Block table — A lookup structure that maps a request's logical sequence of KV cache pages to the physical pages that actually hold that data in GPU memory.

External fragmentation — Wasted memory that occurs when enough total free space exists but it is scattered in gaps too small individually to satisfy a new contiguous allocation.

Chunked prefill — A scheduling strategy that interleaves pieces of a new request's prefill work with ongoing decode steps, instead of finishing prefill entirely before resuming decode.

Time to first token (TTFT) — The delay between sending a prompt and receiving the first generated token back, dominated by the prefill phase.

Decode phase — The memory-bound stage of LLM inference where the model generates one output token at a time, reading the accumulated context from GPU memory at each step.

Autoregressive generation — Generating text one token at a time, where each new token is produced based on all tokens generated so far.

PagedAttention — A memory management technique for KV cache that splits it into small fixed-size, non-contiguous pages addressed through a block table, modeled on OS virtual memory paging.

Internal fragmentation — Wasted memory that occurs when a fixed allocation is sized for the worst case but the actual usage is much smaller, leaving the unused portion locked up and unusable by others.

Prefix caching — Reusing the same stored KV cache pages across multiple requests that share an identical prompt prefix, instead of recomputing and storing it separately for each.

Speculative decoding — A latency optimization where a small draft model proposes several candidate tokens that a larger model verifies in one forward pass, accepting correct guesses and correcting the first wrong one.

vLLM — An open-source LLM inference and serving engine that introduced PagedAttention and provides the tuning options for KV cache management discussed in this lesson.

Explanations are AI-generated. Check anything you intend to rely on.